

UMS9117 工程模式接口使用指南

文档版本
发布日期

V1.2
2024-03-15

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、**UNISOC**、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

紫光展锐（上海）科技有限公司



前言

概述

本文档介绍了 UMS9117 平台工程模式接口使用指南，内容涵盖工程模式介绍、数据类型、API 函数等。

读者对象

本文档主要适用于 Mocom UMS9117 平台的所有软件和硬件开发人员。

缩略语

缩略语	英文全称	中文解释
ADC	Analog to Digital Converter	模数转换器
AoC	Advice of Charge	计费提醒
DTX	Discontinuous Transmission	非连续发射
LCD	Liquid Crystal Display	液晶显示屏
PID	Protocol Identifier	协议标识
RTC	Real Time Clock	实时时钟

符号约定

在本文中可能出现下列符号，每种符号的说明如下。

符号	说明
 说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
 注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
 警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.2	2024-03-15	- 文档名由《Mocor 工程模式接口使用指南》变更为《UMS9117 工程模式接口使用指南》。 - 新增 2.3.10 VERSION_TYPE_E、2.4.6 MNSIM_AOC_T。 - 更新 3 API 函数 章节结构，新增接口函数。 - 优化描述。
V1.1	2020-03-24	修改文档名、优化内容。
V1.0	2019-07-24	第一次正式发布。

关键字

工程模式、暗码、API

目 录

1 概述.....	1
2 数据类型.....	2
2.1 常量.....	2
2.2 类型定义.....	2
2.3 枚举.....	2
2.3.1 AUDIO_DEVICE_MODE_TYPE_E	2
2.3.2 ERR_AUD_E	3
2.3.3 ERR_MNPHONE_CODE_E	4
2.3.4 ERR_MNSMS_CODE_E.....	5
2.3.5 ERR_LCD_E.....	6
2.3.6 ERR_TM_E.....	7
2.3.7 LCD_ID_E	8
2.3.8 MN_PHONE_BAND_TYPE_E.....	8
2.3.9 MN_SMS_CAUSE_E	10
2.3.10 VERSION_TYPE_E	12
2.4 结构体.....	13
2.4.1 AUDIO_TEST_DATA_T	13
2.4.2 APP_MN_NCELL_RSSI_CNF_T.....	14
2.4.3 MN_LAI_T	15
2.4.4 MN_PHONE_CELL_MEAS_T.....	16
2.4.5 MN_PHONE_SCELL_MEAS_T.....	16
2.4.6 MNSIM_AOC_T.....	18
2.4.7 APP_MN_SET_BAND_CNF_T.....	19
2.4.8 APP_MN_SET_SERVICE_TYPE_CNF_T.....	19
2.4.9 SCI_DATE_T.....	20
2.4.10 SCI_TIME_T.....	21
2.5 事件.....	21
2.5.1 EV_MN_APP_SET_BAND_CNF_F.....	21
2.5.2 EV_MN_SET_ASSERT_SMS_DESTADDR_CNF_F	22
3 API 函数	24
3.1 REF 接口函数.....	24
3.1.1 ADC_GetTemperature	24
3.1.2 ADC_GetValue	24
3.1.3 AUD_SetEngineeringParameter.....	25
3.1.4 AUD_GetEngineeringParameter	26
3.1.5 CHR_GetVoltage	26
3.1.6 CHR_GetCurVoltage	27

3.1.7 DCAMERA_GetSensorStr.....	27
3.1.8 LCM_GetCfgInfo.....	28
3.1.9 LCD_SetContrast.....	28
3.1.10 LCD_SetBrightness.....	29
3.1.11 OSPARAM_SetAssertMode.....	30
3.1.12 OSPARAM_GetAssertMode.....	30
3.1.13 REFPARAM_SetDspDebugPortPhyNo.....	31
3.1.14 REFPARAM_GetDspDebugPortPhyNo.....	32
3.1.15 REFPARAM_SetAutoPowerOnFlag.....	32
3.1.16 REFPARAM_GetAutoPowerOnFlag.....	33
3.1.17 REFPARAM_SetDspCardLogFlag.....	33
3.1.18 REFPARAM_GetDspCardLogFlag.....	34
3.1.19 SCI_SetAssertMode.....	34
3.1.20 SCI_SetArmLogFlag.....	35
3.1.21 SCI_GetArmLogFlag.....	36
3.1.22 SCI_ENGINEER_SetEnableArmSleepFlag.....	36
3.1.23 SCI_ENGINEER_GetEnableArmSleepFlag.....	37
3.1.24 SIO_Close.....	37
3.1.25 SIO_ChangeMode.....	38
3.1.26 SIO_SetUserPortMode.....	38
3.1.27 SIO_DSPCardLogOpen.....	39
3.1.28 SIO_DSPCardLogClose.....	40
3.1.29 TM_GetSysDate.....	40
3.1.30 TM_GetSysTime.....	41
3.1.31 UBOT_SetDiskHidden.....	41
3.1.32 UBOT_GetDiskHiddenState.....	42
3.1.33 VERSION_GetInfo.....	43
3.2 MMI 接口函数.....	43
3.2.1 MMIAPISSET_SetTestModeEx.....	43
3.2.2 MMISRVAUD_StartVibrate.....	44
3.2.3 MMISRVAUD_StopVibrate.....	44
3.2.4 MMI_GetAllNVSize.....	45
3.2.5 MMI_GetAllModuleSumNvSize.....	45
3.2.6 MMI_GetAllMMINVItem.....	46
3.2.7 MMIENGARRAY_StrcatFromStrToUCS2.....	46
3.2.8 MMIAPIENG_HandlePlmnCnf.....	47
3.3 MN 接口函数.....	47
3.3.1 DMNV_SetUsimDrvLogFlag.....	47
3.3.2 mn_al_em_set_ims_parameter.....	48
3.3.3 mn_al_em_get_ims_parameter.....	49
3.3.4 MNPHONE_GetTestModeEx.....	49
3.3.5 LTE_Get_UE_Category_Info_Proc.....	50

3.3.6	MNPHONE_GetCurrentLteRat.....	50
3.3.7	MNPHONE_GetCurrentUmtsRat	51
3.3.8	MNPHONE_Get_Fdd_Freqitem	51
3.3.9	MNPHONE_SetLockedFreqAndCell	52
3.3.10	MNPHONE_GetLockedFreqAndCell.....	53
3.3.11	MNPHONE_UnlockedFreqAndCell.....	53
3.3.12	em_set_aoc_nv_param	54
3.3.13	em_get_aoc_nv_param.....	54
3.3.14	MNSIM_GetAocEx.....	55
3.3.15	RRENG_Is_Delay_CSFB_1_Enable_upper	55
3.3.16	RRENG_Is_Delay_CSFB_2_Enable_upper	56
3.3.17	RRENG_Delay_CSFB_Enable	56
3.3.18	RRENG_Delay_CSFB_Disable	57
3.3.19	em_get_fdd_support_band	57
3.3.20	em_al_get_locked_lte_multiband_for_scan	58
3.3.21	em_al_get_locked_lte_multiband.....	58
3.3.22	MNPHONE_Set_LTE_Band	59
3.3.23	MNPHONE_Get_LTE_Band.....	60
3.3.24	MNPHONE_SetRAT_For_Lock.....	60
3.3.25	em_set_fdd_rf_band	61
3.3.26	em_get_fdd_rf_band.....	62
3.3.27	MNPHONE_SelectBandEx.....	62
3.3.28	MNPHONE_GetBandEx.....	63
3.3.29	MNPHONE_GprsAttachEx.....	64
3.3.30	MNPHONE_GprsDetachEx	64
3.3.31	Lte_SetAntennaSingleMode.....	65
3.3.32	Lte_GetAntennaSingleMode	66
3.3.33	Lte_SetAntennaMode	66
3.3.34	Lte_GetAntennaMode	67
3.3.35	MNPHONE_SetServiceTypeEx.....	67
3.3.36	MNPHONE_GetServiceTypeEx	68
3.3.37	MNPHONE_GetBand	69
3.3.38	MNPHONE_SetServiceType	69
3.3.39	MNPHONE_GetServiceType	70
3.3.40	MNNV_GetIMEI	70
3.3.41	MNNV_GetIMEIEx	71
3.3.42	MNPHONE_GetMeasReportEx.....	71
3.3.43	MNPHONE_GetRSSIEx.....	72
3.4	L1 接口函数.....	73
3.4.1	L1API_GetAFC.....	73
3.4.2	L1API_GetPA	73
3.4.3	L1API_GetAGC.....	74

3.4.4 L1API_ApplyMaxPower.....	75
3.4.5 L1API_WriteGsmDivState.....	75
3.4.6 L1API_ReadGsmDivState	76
3.4.7 WL1C_RF_Set_Path_For_ATNA_Test.....	76
3.4.8 WL1C_RF_Get_Path_For_ATNA_Test.....	77
3.4.9 LAYER1_GetLayer1MonItem.....	77
3.4.10 DRV_RF_NV_Support_Diversity_Path.....	78
3.5 TCPIP 接口函数	79
3.5.1 TCPIPDEBUG_EnablePktOutput	79
3.5.2 TCPIPDEBUG_IsPktOutputEnable	79
4 参考文档.....	81

1 概述

设备工程模式的功能包括调试和测试两部分。

调试

- 工程模式下的菜单调试：通过暗码从 IDLE 状态下进入工程模式菜单。在工程模式菜单下，查询和修改设备的参数。
- 暗码调试：在 IDLE 状态下，通过暗码查询和修改设备的参数。

说明

部分接口函数在工程模式与非工程模式下一致，本文未提及的内容可参考对应模块的接口使用指南。

测试

主要是在工厂生产中对设备进行按键、声音、振动等测试。UI 测试包括如下测试项目：

- SIM 卡测试
- T 卡测试
- LCD（Liquid Crystal Display，液晶显示屏）测试
- 听筒测试
- 扬声器测试
- 麦克到听筒测试
- 麦克到扬声器测试
- 照相机测试
- 相片读取测试
- 手电筒测试
- 振动测试
- 键盘灯测试
- 按键测试
- 收音机测试
- 耳机听筒测试
- 耳机麦克到耳机听筒测试
- 蓝牙测试
- 充电测试
- RTC（Real Time Clock，实时时钟）测试

说明

与上述测试项目有关的数据结构及 API 函数定义，可参考对应模块的接口使用指南。

2 数据类型

本章介绍了工程模式所涉及到的各种数据类型，如常量、枚举、结构体等。

2.1 常量

常量名	取值	说明
MN_MAX_MOBILE_ID_LENGTH	9	存放移动标识的最大长度，设备定义为9，模拟器定义为8。
MN_MAX_WNCELL_NUM	96	最大邻近小区数。
MN_SMS_ASSERT_DEST_ADDR_LEN_MAX	8	存放发送 Assert 短消息的目的地址的数组最大长度。

2.2 类型定义

类型名	类型	说明
MN_IMEI_T[MN_MAX_MOBILE_ID_LENGTH]	uint8	定义存放 IMEI 数组结构。
NV_ASSERT_SMS_FLAG_T	uint8	标志是否把设备发生 Assert 时的定位信息以 SMS 消息的方式发送给服务器。
MN_SMS_ASSERT_DEST_ADDR_ARR_T[MN_SMS_ASSERT_DEST_ADDR_LEN_MAX]	uint8	Assert 短消息发送的目的地址。

2.3 枚举

2.3.1 AUDIO_DEVICE_MODE_TYPE_E

【功能】

AUDIO_DEVICE_MODE_TYPE_E 定义了 Audio 设备的模式类型。

【定义】

```
typedef enum
```

```
{
```

```
    AUDIO_DEVICE_MODE_HANDHOLD,
```

```
    AUDIO_DEVICE_MODE_HANDFREE,
```

```
    AUDIO_DEVICE_MODE_EARPHONE,
```

```
    AUDIO_DEVICE_MODE_EARPHONE_3P,
```

```
    AUDIO_DEVICE_MODE_EARFREE,
```

```
    AUDIO_DEVICE_MODE_TVOUT,
```

```
    AUDIO_DEVICE_MODE_BLUEPHONE,
```

```
    AUDIO_DEVICE_MODE_BLUEPHONE_NREC,
```

```
    AUDIO_DEVICE_MODE_HANDFREE_EXT,
```

```
    AUDIO_DEVICE_MODE_MAX,
```

```
    AUDIO_DEVICE_MODE_CUR
```

```
}AUDIO_DEVICE_MODE_TYPE_E;
```

【参数说明】

参数名称	含义
AUDIO_DEVICE_MODE_HANDHOLD	手持模式
AUDIO_DEVICE_MODE_HANDFREE	免提模式
AUDIO_DEVICE_MODE_EARPHONE	耳机模式
AUDIO_DEVICE_MODE_EARPHONE_3P	3P 耳机（三线耳机：左声道、右声道、地线）
AUDIO_DEVICE_MODE_EARFREE	耳机免提模式
AUDIO_DEVICE_MODE_TVOUT	TV 播放模式
AUDIO_DEVICE_MODE_BLUEPHONE	蓝牙耳机
AUDIO_DEVICE_MODE_BLUEPHONE_NREC	支持 NREC 的蓝牙耳机
AUDIO_DEVICE_MODE_HANDFREE_EXT	系统预留，当前未使用
AUDIO_DEVICE_MODE_MAX	系统保留，不可使用
AUDIO_DEVICE_MODE_CUR	当前模式

2.3.2 ERR_AUD_E

【功能】

ERR_AUD_E 定义了 Audio 模块接口函数返回的错误代码。

【定义】

```
typedef enum
{
    ERR_AUD_NONE,
    ERR_AUD_PARAM,
    ERR_AUD_CONFLICT,
    ERR_AUD_MAX
} ERR_AUD_E;
```

【参数说明】

参数名称	含义
ERR_AUD_NONE	无错误
ERR_AUD_PARAM	输入参数错误
ERR_AUD_CONFLICT	正在播放 Ring 或 MIDI，无法处理 Tone
ERR_AUD_MAX	系统保留

2.3.3 ERR_MNPHONE_CODE_E

【功能】

ERR_MNPHONE_CODE_E 定义了 MN Phone 模块接口函数返回的错误代码。

【定义】

```
typedef enum _ERR_MNPHONE_CODE_E
{
    ERR_MNPHONE_NO_ERR = 0,
    ERR_MNPHONE_PTR_IS_NULL = (0x0030 << 16 + 0),
    ERR_MNPHONE_NOT_RIGHT_PARAM,
    ERR_MNPHONE_OUT_OF_RANGE,
    ERR_MNPHONE_WRITE_ME_FAILURE
    ERR_MNPHONE_OPERATE_NOT_ALLOWED
} ERR_MNPHONE_CODE_E;
```

【参数说明】

参数名称	含义
ERR_MNPHONE_NO_ERR	调用 API 函数成功

参数名称	含义
ERR_MNPHONE_PTR_IS_NULL	指针指向空值
ERR_MNPHONE_NOT_RIGHT_PARAM	填写的参数不符合正常情况
ERR_MNPHONE_OUT_OF_RANGE	长度超出了范围
ERR_MNPHONE_WRITE_ME_FAILURE	写参数到 NV 中失败
ERR_MNPHONE_OPERATE_NOT_ALLOWED	操作不允许

2.3.4 ERR_MNSMS_CODE_E

【功能】

ERR_MNSMS_CODE_E 定义了 SMS 模块接口函数返回的错误代码。

【定义】

```
typedef enum _ERR_MNSMS_CODE_E
{
    ERR_MNSMS_NONE = 0,
    ERR_MNSMS_OUT_OF_RANGE = (0x0010 << 16 + 0),
    ERR_MNSMS_NOT_RIGHT_STORAGE,
    ERR_MNSMS_SIM_NOT_READY,
    ERR_MNSMS_NOT_RIGHT_PARAM,
    ERR_MNSMS_SIM_SET_ERROR,
    ERR_MNSMS_NOT_RIGHT_STATUS,
    ERR_MNSMS_PTR_IS_NULL,
    ERR_MNSMS_ME_NOT_SUPPORT,
    ERR_MNSMS_NO_SC_ADDRESS,
    ERR_MNSMS_NO_DEST_ADDRESS,
    ERR_MNSMS_NO_PID,
    ERR_MNSMS_NO_VAILID_TIME,
    ERR_MNSMS_WRITE_ME_FAILURE
    ERR_MNSMS_ME_NOT_READY
} ERR_MNSMS_CODE_E;
```

【参数说明】

参数名称	含义
ERR_MNSMS_NONE	调用 API 函数成功

参数名称	含义
ERR_MNSMS_OUT_OF_RANGE	短消息内容（user data 中的数据）的长度超出了范围，分为三种情况： - 如果是没有压缩过的 8-bit 的数据，TP-UD 即短消息内容的最大长度是 134（140-6）个字节 - 如果是没有压缩过的 Default 7-bit 的数据，TP-UD 即短消息内容的最大长度是 153（160-7）个 7-bit 字符 - 如果是没有压缩过的 UCS2 的数据，TP-UD 即短消息内容的最大长度是 67（（140-6）/2）个 16-bit 字符
ERR_MNSMS_NOT_RIGHT_STORAGE	填写的存储位置不正确，不是 SIM 卡或者 ME 中的一种
ERR_MNSMS_SIM_NOT_READY	SIM 卡未准备好
ERR_MNSMS_NOT_RIGHT_PARAM	填写的参数不符合正常情况
ERR_MNSMS_SIM_SET_ERROR	SIM 卡设置错误
ERR_MNSMS_NOT_RIGHT_STATUS	填写的状态不正确，不是 MN_SMS_DELETE_ALL_STATUS_E 或者 MN_SMS_STATUS_E 中定义的正确值
ERR_MNSMS_PTR_IS_NULL	指针指向空值
ERR_MNSMS_ME_NOT_SUPPORT	不支持存储到 ME 中
ERR_MNSMS_NO_SC_ADDRESS	没有短消息中心号码
ERR_MNSMS_NO_DEST_ADDRESS	没有目的地址
ERR_MNSMS_NO_PID	发送短消息时没有 PID（Protocol Identifier，协议标识）
ERR_MNSMS_NO_VAILID_TIME	发送短消息时没有填入有效时间
ERR_MNSMS_WRITE_ME_FAILURE	写参数到 ME 中失败
ERR_MNSMS_ME_NOT_READY	ME 未准备好

2.3.5 ERR_LCD_E

【功能】

ERR_LCD_E 定义了 LCD 模块接口函数返回的错误代码。

【定义】

```
typedef enum
{
  ERR_LCD_NONE = SCI_SUCCESS,
  ERR_LCD_FUNC_NOT_SUPPORT,
```

```
ERR_LCD_PARAMETER_WRONG,
ERR_LCD_POINTER_NULL,
ERR_LCD_OPERATE_FAIL,
ERR_LCD_NOT_FOUND
ERR_LCD_SELPIN_WRONG
ERR_LCD_SPEC_WRONG
} ERR_LCD_E;
```

【参数说明】

参数名称	含义
ERR_LCD_NONE	无错误
ERR_LCD_FUNC_NOT_SUPPORT	该 LCD 不支持该操作
ERR_LCD_PARAMETER_WRONG	输入参数错误
ERR_LCD_POINTER_NULL	指针为空
ERR_LCD_OPERATE_FAIL	操作失败
ERR_LCD_NOT_FOUND	未发现 LCD
ERR_LCD_SELPIN_WRONG	当前未使用
ERR_LCD_SPEC_WRONG	当前未使用

2.3.6 ERR_TM_E

【功能】

ERR_TM_E 定义了 Time 模块接口函数返回的错误代码。

【定义】

```
typedef enum
{
ERR_TM_NONE,
ERR_TM_PNULL,
ERR_TM_WRONG_DATE,
ERR_TM_WRONG_TIME,
ERR_TM_ALARM_WRONG_TIME,
ERR_TM_ALARM_TIME_IS_EXPIRED,
ERR_TM_MAX
} ERR_TM_E;
```

【参数说明】

参数名称	含义
ERR_TM_NONE	无错误
ERR_TM_PNULL	指针为空
ERR_TM_WRONG_DATE	日期错误
ERR_TM_WRONG_TIME	时间错误
ERR_TM_ALARM_WRONG_TIME	闹铃时间错误
ERR_TM_ALARM_TIME_IS_EXPIRED	待设置的闹铃时间已过
ERR_TM_MAX	系统保留，不可使用

2.3.7 LCD_ID_E

【功能】

LCD_ID_E 定义了 LCD 编号。

【定义】

```
typedef enum
{
    MAIN_LCD_ID = 0,
    SUB_LCD_ID,
    MAX_LCD_ID
} LCD_ID_E;
```

【参数说明】

参数名称	含义
MAIN_LCD_ID	LCD 主屏 ID 号，固定为 0
SUB_LCD_ID	LCD 副屏 ID 号
MAX_LCD_ID	系统保留，不可使用

2.3.8 MN_PHONE_BAND_TYPE_E

【功能】

MN_PHONE_BAND_TYPE_E 定义了设备的工作频段类型。

【定义】

```
typedef uint8 MN_PHONE_BAND_TYPE_E;
#define MN_PHONE_BAND_GSM 0x00
#define MN_PHONE_BAND_DCS 0x01
#define MN_PHONE_BAND_GSM_DCS 0x02
#define MN_PHONE_BAND_PCS 0x03
#define MN_PHONE_BAND_GSM850 0x04
#define MN_PHONE_BAND_GSM_PCS 0x05
#define MN_PHONE_BAND_GSM850_DCS 0x06
#define MN_PHONE_BAND_GSM850_PCS 0x07
#define MN_PHONE_BAND_GSM850_GSM 0x08
#define MN_PHONE_BAND_GSM850_GSM_PCS 0x09
#define MN_PHONE_BAND_GSM850_GSM_DCS 0x0a
#define MN_PHONE_BAND_GSM_DCS_PCS 0x0b
#define MN_PHONE_BAND_GSM850_GSM_DCS_PCS 0x0c
#define MN_PHONE_BAND_DCS_PCS 0x0d
#define MN_PHONE_BAND_GSM850_DCS_PCS 0x0e
#define MN_PHONE_BAND_MAX_NUM 0x10
```

【参数说明】

参数名称	含义
MN_PHONE_BAND_GSM	GSM 频段
MN_PHONE_BAND_DCS	DCS 频段
MN_PHONE_BAND_GSM_DCS	GSM_DCS 频段
MN_PHONE_BAND_PCS	PCS 频段
MN_PHONE_BAND_GSM850	GSM850 频段
MN_PHONE_BAND_GSM_PCS	GSM_PCS 频段
MN_PHONE_BAND_GSM850_DCS	GSM850_DCS 频段
MN_PHONE_BAND_GSM850_PCS	GSM850_PCS 频段
MN_PHONE_BAND_GSM850_GSM	GSM850_GSM 频段
MN_PHONE_BAND_GSM850_GSM_PCS	GSM850_GSM_PCS 频段
MN_PHONE_BAND_GSM850_GSM_DCS	GSM850_GSM_DCS 频段
MN_PHONE_BAND_GSM_DCS_PCS	GSM_DCS_PCS 频段
MN_PHONE_BAND_GSM850_GSM_DCS_PCS	GSM850_GSM_DCS_PCS 频段

参数名称	含义
MN_PHONE_BAND_DCS_PCS	DCS_PCS 频段
MN_PHONE_BAND_GSM850_DCS_PCS	GSM850_DCS_PCS 频段
MN_PHONE_BAND_MAX_NUM	频段最大数目

2.3.9 MN_SMS_CAUSE_E

【功能】

MN_SMS_CAUSE_E 定义了短消息操作结果类型。

【定义】

```
typedef enum _MN_SMS_CAUSE_E
{
    MN_SMS_OPERATE_SUCCESS,
    MN_SMS_UNSPEC_ERROR,
    MN_SMS_SEND_FAILURE,
    MN_SMS_DEFAULT_PARAM_ERROR,
    MN_SMS_MN_BUSY,
    MN_SMS_WRITE_NV_FAILED,
    MN_SMS_ME_MEM_FULL,
    MN_SMS_ME_NOT_SUPPORT,
    MN_SMS_ME_DELETE_FAILURE,
    MN_SMS_SIM_NOT_READY,
    MN_SMS_SIM_MEM_FULL,
    MN_SMS_SIM_PATTERN_NOT_FIND,
    MN_SMS_SIM_SEEK_FAILED,
    MN_SMS_SIM_ACCESS_FAILED,
    MN_SMS_SIM_READ_FAILED,
    MN_SMS_SIM_SELECT_FAILED,
    MN_SMS_SIM_UPDATE_FAILED,
    MN_SMS_SIM_PURGE_FAILED,
    MN_SMS_SIM_SERVICE_DISABLED,
    MN_SMS_SIM_DELETE_FAILURE,
    MN_SMS_RECORD_OUT_OF_RANGE,
    MN_SMS_NOT_RIGHT_STORAGE,
```

```

MN_SMS_SIM_SAVE_STATUS_FREE,
MN_SMS_SIM_STK_NOT_ALLOWED,
MN_SMS_SIM_UNKNOW_FAILED,
MN_SMS_SIM_FDN_FAILED, /* cause of FDN check failure */
MN_SMS_CANCEL_SUCCESS /* cause of cancel mo sms success*/
} MN_SMS_CAUSE_E;

```

【参数说明】

参数名称	含义
MN_SMS_OPERATE_SUCCESS	操作成功
MN_SMS_UNSPEC_ERROR	操作产生未知错误
MN_SMS_SEND_FAILURE	短消息发送失败
MN_SMS_DEFAULT_PARAM_ERROR	短消息的缺省参数错误
MN_SMS_MN_BUSY	MN 层处于忙状态，不能进行用户请求的操作
MN_SMS_WRITE_NV_FAILED	写 NV 失败
MN_SMS_ME_MEM_FULL	ME 存储区已满
MN_SMS_ME_NOT_SUPPORT	不支持 ME
MN_SMS_ME_DELETE_FAILURE	从 ME 中删除失败
MN_SMS_SIM_NOT_READY	SIM 卡没有准备好
MN_SMS_SIM_MEM_FULL	SIM 卡的存储区已满
MN_SMS_SIM_PATTERN_NOT_FIND	不能在 SIM 卡中找到相应的情况
MN_SMS_SIM_SEEK_FAILED	查找 SIM 卡失败
MN_SMS_SIM_ACCESS_FAILED	没有权限读取 SIM 卡
MN_SMS_SIM_READ_FAILED	读 SIM 卡文件出错
MN_SMS_SIM_SELECT_FAILED	选择 SIM 卡文件失败
MN_SMS_SIM_UPDATE_FAILED	更新 SIM 卡文件失败
MN_SMS_SIM_PURGE_FAILED	清除 SIM 卡文件失败
MN_SMS_SIM_SERVICE_DISABLED	请求的 SIM 卡服务是失效的
MN_SMS_SIM_DELETE_FAILURE	从 SIM 卡中删除失败
MN_SMS_RECORD_OUT_OF_RANGE	Record ID 超出范围
MN_SMS_NOT_RIGHT_STORAGE	存储器不正确
MN_SMS_SIM_SAVE_STATUS_FREE	保存长短消息，保存返回成功，但读时显示为空白

参数名称	含义
MN_SMS_SIM_STK_NOT_ALLOWED	STK 不允许
MN_SMS_SIM_UNKNOW_FAILED	SIM 未知错误
MN_SMS_SIM_FDN_FAILED	FDN 确认失败
MN_SMS_CANCEL_SUCCESS	取消 SMS 成功

2.3.10 VERSION_TYPE_E

【功能】

获取版本信息的类型。

【定义】

```
typedef enum
```

```
{  
    PLATFORM_VERSION,  
    PROJECT_VERSION,  
    MMI_VERSION,  
    BASE_VERSION,  
    HW_VERSION,  
    CALIBRATION_VERSION,  
    DOWNLOAD_VERSION,  
    BUILD_TIME,  
    LOG_VERSION,  
    MEMORY_NAME,  
    CHIP_NAME,  
    SVN_VERSION,  
    ROOT_SVN_VERSION,  
    EXTERNALS_SVN_VERSION,  
    MAX_VERSION_NUM,  
}VERSION_TYPE_E;
```

【参数说明】

参数名称	含义
PLATFORM_VERSION	平台版本
PROJECT_VERSION	项目版本

参数名称	含义
MMI_VERSION	MMI 版本
BASE_VERSION	Modem 版本
HW_VERSION	硬件版本
CALIBRATION_VERSION	校准版本，当前为空
DOWNLOAD_VERSION	下载版本，当前为空
BUILD_TIME	构建时间
LOG_VERSION	Log 版本
MEMORY_NAME	内存名称
CHIP_NAME	芯片名称
SVN_VERSION	SVN 版本
ROOT_SVN_VERSION	AP 侧 SVN 版本
EXTERNALS_SVN_VERSION	外链 SVN 版本
MAX_VERSION_NUM	版本枚举最大值

2.4 结构体

2.4.1 AUDIO_TEST_DATA_T

【功能】

AUDIO_TEST_DATA_T 定义了 Audio 测试参数的数据结构。

【定义】

```
typedef struct
```

```
{
```

```
int16    UL_equalizer_coeff[33];
```

```
int16    DL_equalizer_coeff[33];
```

```
int16    UL_PGA_gain;
```

```
int16    DL_PGA_gain;
```

```
int16    UL_digital_gain;
```

```
int16    DL_digital_gain;
```

```
int16    UL_digital_scale;
```

```
int16    DL_digital_scale;
int16    midi_PGA_gain_base;
int16    Digital_sidetone_gain;
int16    DL_IIR1_coeff[6];
int16    DL_IIR2_coeff[6];
int16    reserved[2];
} AUDIO_TEST_DATA_T;
```

【参数说明】

参数名称	含义	取值
UL_equalizer_coeff[33]	上行 FIR 均衡器系数	0 ~ 32768
DL_equalizer_coeff[33]	下行 FIR 均衡器系数	0 ~ 32768
UL_PGA_gain	上行模拟增益	0 ~ 11
DL_PGA_gain	下行模拟增益	0 ~ 11
UL_digital_gain	上行数字增益	0 ~ 32768
DL_digital_gain	下行数字增益	0 ~ 32768
UL_digital_scale	上行数字补偿增益	0 ~ 32768
DL_digital_scale	下行数字补偿增益	0 ~ 32768
midi_PGA_gain_base	播放 MIDI 时模拟增益基准值	0 ~ 11
Digital_sidetone_gain	数字侧音增益	0 ~ 32768
DL_IIR1_coeff[6]	下行 IIR-1 均衡器系数	0 ~ 32768
DL_IIR2_coeff[6]	下行 IIR-2 均衡器系数	0 ~ 32768
reserved[2]	系统保留	-

2.4.2 APP_MN_NCELL_RSSI_CNF_T

【功能】

APP_MN_NCELL_RSSI_CNF_T 为消息结构，对应的消息为 APP_MN_NCELL_RSSI_CNF，定义了临近小区的信息。

【定义】

```
typedef struct
{
    SIGNAL_VARS
```

```

MN_PHONE_CELL_MEAS_T   ncell_power[MN_MAX_WNCELL_NUM];
MN_DUAL_SYS_E           dual_sys;
} APP_MN_NCELL_RSSI_CNF_T;
    
```

【参数说明】

参数名称	含义
SIGNAL_VARS	消息头
ncell_power	该数组用来保存所有临近小区的信息
dual_sys	指定当前使用的 SIM 卡 • 0: 卡 1 • 1: 卡 2

2.4.3 MN_LAI_T

【功能】

MN_LAI_T 定义了位置区识别码结构。

【定义】

```

typedef struct MN_LAI_T
{
    uint16  mcc;
    uint16  mnc;
    uint16  mnc_digit_num;
    uint16  lac;
} MN_LAI_T;
    
```

【参数说明】

参数名称	含义
mcc	移动国家码（Mobile Country Code）
mnc	移动网络码（Mobile Network Code）
mnc_digit_num	移动国家码位数
lac	位置区码（Location Area Code）

2.4.4 MN_PHONE_CELL_MEAS_T

【功能】

MN_PHONE_CELL_MEAS_T 定义了小区的基本信息。

【定义】

```
typedef struct MN_PHONE_CELL_MEAS_T
```

```
{
    BOOLEAN      cell_present;
    uint16       arfcn;
    uint8        bsic;
    uint8        rxlev;
    BOOLEAN      identity_present;
    MN_LAI_T     lai;
    uint16       cell_id;
} MN_PHONE_CELL_MEAS_T;
```

【参数说明】

参数名称	含义
cell_present	小区是否存在 - TRUE: 存在 - FALSE: 不存在
arfcn	小区频点, 仅当 cell_present 为 TRUE 时有效
bsic	小区色码, 仅当 cell_present 为 TRUE 时有效
rxlev	小区信号强度, 仅当 cell_present 为 TRUE 时有效
identity_present	小区是否已解析成功 - TRUE: 已解析成功 - FALSE: 未解析成功
lai	小区位置信息, 仅当 identity_present 为 TRUE 时有效
cell_id	小区 ID, 仅当 identity_present 为 TRUE 时有效

2.4.5 MN_PHONE_SCELL_MEAS_T

【功能】

MN_PHONE_SCELL_MEAS_T 定义了服务小区的基本信息。

【定义】

```
typedef struct MN_PHONE_SCELL_MEAS_T
{
    uint16 arfcn;
    uint8 bsic;
    BOOLEAN is_traffic_mode;
    // idle info
    int16 rxlev;
    // traffic info
    BOOLEAN is_dtx_used;
    uint16 rxqual_full;
    uint16 rxqual_sub;
    uint16 rxlev_full;
    uint16 rxlev_sub;
    uint32 cell_id;
    uint16 lac;
    MN_SYS_SIM_E dual_sys;
    uint32 physical_cell_id;
    uint32 band;
} MN_PHONE_SCELL_MEAS_T;
```

【参数说明】

参数名称	含义
arfcn	服务小区的频点
bsic	小区色码
is_traffic_mode	是否在通话状态 - TRUE: 是 - FALSE: 否
rxlev	服务小区信号强度
is_dtx_used	是否为 DTX (Discontinuous Transmission, 非连续发射) 模式, - TRUE: 是 - FALSE: 否 仅当 is_traffic_mode 为 TRUE 时有效
rxqual_full	主天线的通话质量, 仅当 is_traffic_mode 为 TRUE 时有效
rxqual_sub	辅天线的通话质量, 仅当 is_traffic_mode、is_dtx_used 均为 TRUE 时有效

参数名称	含义
rxlev_full	主天线的信号强度，仅当 is_traffic_mode 为 TRUE 时有效
rxlev_sub	辅天线的信号强度，仅当 is_traffic_mode、is_dtx_used 均为 TRUE 时有效
cell_id	当前的小区编号
lac	当前的位置区编号
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2
physical_cell_id	物理小区 ID
band	频段

2.4.6 MNSIM_AOC_T

【功能】

AoC（Advice of Charge，计费提醒）计费相关信息。

【定义】

```
typedef struct MNSIM_AOC_T
```

```
{
```

```
    BOOLEAN    aoc_is_support;
```

```
    uint32     acm_unit;
```

```
    uint32     acm_max;
```

```
    MNSIM_PUCT_INFO_T puct;
```

```
    uint32     line2_acm_unit;
```

```
} MNSIM_AOC_T;
```

【参数说明】

参数名称	含义
aoc_is_support	是否支持 AoC 功能 - TRUE: 支持 - FALSE: 不支持
acm_unit	当前 AoC 计费值
acm_max	SIM 卡中存储的 AoC 计费允许最大值
puct	目前未赋有效值，全 0

参数名称	含义
line2_acm_unit	目前未赋有效值，0

2.4.7 APP_MN_SET_BAND_CNF_T

【功能】

APP_MN_SET_BAND_CNF_T 为消息结构，对应的消息为 APP_MN_SET_BAND_CNF，通知用户设置频段是否成功。

【定义】

```
typedef struct
{
    SIGNAL_VARS
    BOOLEAN is_ok;
    MN_DUAL_SYS_E dual_sys;
} APP_MN_SET_BAND_CNF_T;
```

【参数说明】

参数名称	含义
SIGNAL_VARS	消息头
is_ok	更改频段的操作是否成功 - TRUE：操作成功 - FALSE：操作失败
dual_sys	指定当前使用的 SIM 卡 0：卡 1 1：卡 2

2.4.8 APP_MN_SET_SERVICE_TYPE_CNF_T

【功能】

APP_MN_SET_SERVICE_TYPE_CNF_T 为消息结构，对应的消息为 APP_MN_SET_SERVICE_TYPE_CNF，通知用户设置服务种类是否成功。

【定义】

```
typedef struct
```

```
{
    SIGNAL_VARS
    BOOLEAN    is_ok;
    MN_DUAL_SYS_E    dual_sys;
} APP_MN_SET_SERVICE_TYPE_CNF_T;
```

【参数说明】

参数名称	含义
SIGNAL_VARS	消息头
is_ok	设置服务种类的操作是否成功 - TRUE: 操作成功 - FALSE: 操作失败
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2

2.4.9 SCI_DATE_T

【功能】

SCI_DATE_T 定义了系统日期结构。

【定义】

```
typedef struct
{
    uint8    mday;
    uint8    mon;
    uint16   year;
    uint8    wday;
} SCI_DATE_T;
```

【参数说明】

参数名称	含义	取值
mday	日	取值范围和具体的月份有关, [1, 28]或[1, 29]或[1, 30]或[1, 31]
mon	月份	1 ~ 12
year	年份	2000 ~ 2049

参数名称	含义	取值
wday	星期	0~6。其中：0 为星期日；1 为星期一

2.4.10 SCI_TIME_T

【功能】

SCI_TIME_T 定义了系统时间结构。

【定义】

```
typedef struct
{
    uint8      sec;
    uint8      min;
    uint8      hour;
} SCI_TIME_T;
```

【参数说明】

参数名称	含义	取值
sec	秒	0 ~ 59
min	分	0 ~ 59
hour	时	0 ~ 23

2.5 事件

2.5.1 EV_MN_APP_SET_BAND_CNF_F

【功能】

通知用户设置频段是否成功。

该事件用于通知用户设置频段是否成功。如果用户没有提供相应的 CALLBACK 函数，Server 会发送缺省的消息 APP_MN_SET_BAND_CNF，该消息的结构为 APP_MN_SET_BAND_CNF_T。

【触发条件】

由于调用 MNPHONE_SelectBand 导致。

【消息方式】

- 消息名：APP_MN_SET_BAND_CNF
- 消息结构：APP_MN_SET_BAND_CNF_T，包括对设置结果的描述。

【回调方式】

回调函数参数 param_ptr 指向的内存块结构为 APP_MN_SET_BAND_CNF_T；参数 event_id 为 EV_MN_APP_SET_BAND_CNF_F。

【消息结构定义】

```
typedef struct
{
    SIGNAL_VARS
    BOOLEAN is_ok;
    MN_DUAL_SYS_E dual_sys;
} APP_MN_SET_BAND_CNF_T;
```

【参数说明】

参数名称	含义
SIGNAL_VARS	消息头
is_ok	更改频段的操作是否成功。 • TRUE：操作成功 • FALSE：操作失败
dual_sys	指定当前使用的 SIM 卡 • 0：卡 1 • 1：卡 2

2.5.2 EV_MN_SET_ASSERT_SMS_DESTADDR_CNF_F

【功能】

汇报设置 Assert 短消息目的地址结果。

该事件用于汇报设置 Assert 短消息目的地址的操作结果。如果用户没有提供相应的 CALLBACK 函数，Server 会发送缺省的消息 APP_MN_SET_ASSERT_SMS_DESTADDR_CNF，该消息的结构为 APP_MN_SET_ASSERT_SMS_DESTADDR_CNF_T。

【触发条件】

- 由于设置 Assert 短消息目的地址的操作引起。
- 由于调用 MNSMS_SetAssertSmsDestAddr 导致的汇报设置 Assert 短消息目的地址的操作结果。

【消息方式】

- 消息名：APP_MN_SET_ASSERT_SMS_DESTADDR_CNF
- 消息结构：APP_MN_SET_ASSERT_SMS_DESTADDR_CNF_T，汇报设置 Assert 短消息目的地址的操作结果。

【回调方式】

回调函数参数 param_ptr 指向的内存块结构为 APP_MN_SET_ASSERT_SMS_DESTADDR_CNF_T；参数 event_id 为 EV_MN_SET_ASSERT_SMS_DESTADDR_CNF_F。

【消息结构定义】

```
typedef struct
{
    SIGNAL_VARS
    MN_SMS_CAUSE_E    cause;
    MN_DUAL_SYS_E     dual_sys;
} APP_MN_SET_ASSERT_SMS_DESTADDR_CNF_T;
```

【参数说明】

参数名称	含义
SIGNAL_VARS	消息头
cause	Assert 原因
dual_sys	指定当前使用的 SIM 卡 • 0：卡 1 • 1：卡 2

3

API 函数

本章详细描述了工程模式包含的 API 函数的定义及相关说明。

3.1 REF 接口函数

3.1.1 ADC_GetTemperature

【函数功能】

ADC（Analog to Digital Converter，模数转换器）温度查询。调用此 API 函数即可获得当前温度。

【函数原型】

```
uint32 ADC_GetTemperature (void);
```

【参数说明】

无

【返回值】

当前温度。最低三位为小数，如返回值为 3420，则表示 3.420°C。

【注意事项】

获得的值可能不正确，需要硬件调试。

3.1.2 ADC_GetValue

【函数功能】

获取 ADC 耳机按键音量大小值。调用此 API 函数即可获得耳机按键的音量大小。

【函数原型】

```
uint32 ADC_GetValue (  
uint16 channel  
);
```

【参数说明】

参数名称	含义
channel	ADC 通道号，取值范围为 0 ~ 4

【返回值】

ADC 的转换结果，取值范围为 0 ~ 1023。若发生错误，则返回 0xFFFFFFFF。

【注意事项】

无

3.1.3 AUD_SetEngineeringParameter

【函数功能】

设置 Audio 工程参数。

【函数原型】

```
ERR_AUD_E AUD_SetEngineeringParameter (
    AUDIO_DEVICE_MODE_TYPE_E    parameter_type,
    AUDIO_TEST_DATA_T    parameter_data
);
```

【参数说明】

参数名称	含义
parameter_type	详细信息请参考 2.3.1 AUDIO_DEVICE_MODE_TYPE_E 定义
parameter_data	详细信息请参考 2.4.1 AUDIO_TEST_DATA_T 定义

【返回值】

- ERR_AUD_NONE：调用成功。
- 其他：调用失败。

详细信息请参考 [2.3.2 ERR_AUD_E](#) 定义。

【注意事项】

无

3.1.4 AUD_GetEngineeringParameter

【函数功能】

获取 Audio 工程参数。

【函数原型】

```
ERR_AUD_E    AUD_GetEngineeringParameter (
AUDIO_DEVICE_MODE_TYPE_E    parameter_type,
AUDIO_TEST_DATA_T    *parameter_data
);
```

【参数说明】

参数名称	含义
parameter_type	详细信息请参考 2.3.1 AUDIO_DEVICE_MODE_TYPE_E 定义
parameter_data	详细信息请参考 2.4.1 AUDIO_TEST_DATA_T 定义

【返回值】

- ERR_AUD_NONE: 调用成功。
- 其他: 调用失败。

详细信息请参考 [2.3.2 ERR_AUD_E](#) 定义。

【注意事项】

无

3.1.5 CHR_GetVoltage

【函数功能】

ADC 电池平均电压查询。

调用此 API 函数即可获得当前电池的电压，该电压是一个经过处理后的平均值。

【函数原型】

```
uint32    CHR_GetVoltage (void);
```

【参数说明】

无

【返回值】

以 mV 为单位的当前平均电压。如返回值为 4200，则表示 4.20V。

【注意事项】

无

3.1.6 CHR_GetCurVoltage

【函数功能】

ADC 电池当前电压查询。

调用此 API 函数即可获得当前电池的瞬时电压。

【函数原型】

```
uint32 CHR_GetCurVoltage (void);
```

【参数说明】

无

【返回值】

以 mV 为单位的当前瞬时电压。如返回值为 4200，则表示 4.20V。

【注意事项】

无

3.1.7 DCAMERA_GetSensorStr

【函数功能】

获取 sensor name。

【函数原型】

```
PUBLIC uint32 DCAMERA_GetSensorStr(const uint8** sensor_str)
```

【参数说明】

参数名称	含义
sensor_str	返回设备名称

【返回值】

- 0: 成功
- 非 0: 失败

【注意事项】

成功时返回当前使用的 sensor name，具体值由 sensor 驱动配置，例如 GC032A。

3.1.8 LCM_GetCfgInfo

【函数功能】

获取 LCM 的配置信息。

【函数原型】

```
PUBLIC LCM_CFG_INFO_T_PTR LCM_GetCfgInfo (uint32 lcd_id, uint32 *count)
```

【参数说明】

参数名称	含义
lcd_id	• 0: 主屏 • 1: 副屏
count	配置信息个数

【返回值】

返回首个配置信息地址。

【注意事项】

无

3.1.9 LCD_SetContrast

【函数功能】

调节 LCD 对比度。

【函数原型】

```
ERR_LCD_E LCD_SetContrast (
    LCD_ID_E    lcd_id,
    uint16      contrast
```

);

【参数说明】

参数名称	含义
lcd_id	LCD ID，详细信息请参考 2.3.7 LCD_ID_E 定义。
contrast	对比度值，不同的 LCD 支持不同的取值范围。 调用 LCD_GetInfo 可以获取 LCD 对比度的最大值、最小值以及缺省值。

【返回值】

- ERR_LCD_NONE：设置对比度成功。
- 其他：设置失败。

详细信息请参考 [2.3.5 ERR_LCD_E](#) 定义。

【注意事项】

无

3.1.10 LCD_SetBrightness

【函数功能】

调节 LCD 亮度。

【函数原型】

```
ERR_LCD_E LCD_SetBrightness (
LCD_ID_E   lcd_id,
uint16     brightness
);
```

【参数说明】

参数名称	含义
lcd_id	LCD ID，详细信息请参考 2.3.7 LCD_ID_E 定义
brightness	亮度值

【返回值】

- ERR_LCD_NONE：设置亮度成功。
- 其他：设置失败。

详细信息请参考 [2.3.5 ERR_LCD_E](#) 定义。

【注意事项】

不同的 LCD 支持不同的取值范围，具体可以参考每个 LCD 硬件的说明。

3.1.11 OSPARAM_SetAssertMode

【函数功能】

设置系统 Assert mode 为 SCI_ASSERT_DEBUG_MODE 或 SCI_ASSERT_RELEASE_MODE。

【函数原型】

```
PUBLIC uint32 OSPARAM_SetAssertMode(
uint8  assert_mode
)
```

【参数说明】

参数名称	含义
assert_mode	- SCI_ASSERT_DEBUG_MODE 的值为 1，此种 mode 代表系统支持 dump，即死机后会绿屏。 - SCI_ASSERT_RELEASE_MODE 的值为 0x35，此种 mode 代表系统不支持 dump，即死机后会重启。

【返回值】

- SCI_SUCCESS：设置系统模式成功。
- SCI_ERROR：设置系统模式失败。

【注意事项】

此接口的入参为 SCI_ASSERT_DEBUG_MODE 或 SCI_ASSERT_RELEASE_MODE，其他值将会直接返回 SCI_ERROR。

3.1.12 OSPARAM_GetAssertMode

【函数功能】

获取系统 Assert mode。

【函数原型】

```
PUBLIC uint32 OSPARAM_GetAssertMode(
uint8  *assert_mode_ptr
```

)

【参数说明】

参数名称	含义
assert_mode_ptr	用于保存获取的 Assert mode

【返回值】

- SCI_SUCCESS: 获取系统模式成功。
- SCI_ERROR: 获取系统模式失败。

【注意事项】

此接口的入参为一个指向 `uint8` 的指针变量，需注意不能为 `NULL`，否则会直接返回 `SCI_ERROR`。

3.1.13 REFPARAM_SetDspDebugPortPhyNo

【函数功能】

设置 DSP Log 输出方式。

【函数原型】

```
PUBLIC void REFPARAM_SetDspDebugPortPhyNo(  
uint8 phy_port  
)
```

【参数说明】

参数名称	含义
phy_port	• 低四位为 0: 代表不输出 • 低四位为 1: 代表 UART 输出 • 低四位为 2: 代表 USB 输出

【返回值】

无

【注意事项】

具体行为需 DSP 侧配合说明。

3.1.14 REFPARAM_GetDspDebugPortPhyNo

【函数功能】

获取 DSP Log 输出方式。

【函数原型】

```
PUBLIC uint8 REFPARAM_GetDspDebugPortPhyNo(void)
//Returns the dsp log setting
//Low 4-bit is DSP LOG MODE: 0.Disable 1.COM1 2.ARM COM_DEBUG
```

【参数说明】

无

【返回值】

phy_port

- 低四位为 0: 代表不输出
- 低四位为 1: 代表 UART 输出
- 低四位为 2: 代表 USB 输出

【注意事项】

具体行为需 DSP 侧配合说明。

3.1.15 REFPARAM_SetAutoPowerOnFlag

【函数功能】

打开、关闭上电自动开机功能。

【函数原型】

```
void REFPARAM_SetAutoPowerOnFlag (
BOOLEAN   b_auto_power_on
);
```

【参数说明】

参数名称	含义
b_auto_power_on	• TRUE: 打开上电自动开机功能 • FALSE: 关闭上电自动开机功能

【返回值】

无

【注意事项】

无

3.1.16 REFPARAM_GetAutoPowerOnFlag

【函数功能】

获取自动开机功能开关状态。

【函数原型】

```
PUBLIC BOOLEAN REFPARAM_GetAutoPowerOnFlag(void)
```

【参数说明】

无

【返回值】

上电自动开机功能开关标志。

- TRUE: 打开。
- FALSE: 关闭。

【注意事项】

无

3.1.17 REFPARAM_SetDspCardLogFlag

【函数功能】

设置 DSP Log 是否保存到 SD 卡。

【函数原型】

```
PUBLIC void REFPARAM_SetDspCardLogFlag(  
    BOOLEAN is_enable_dsp_card_log  
)
```

【参数说明】

参数名称	含义
is_enable_dsp_card_log	- 高四位为 1：代表保存到 SD 卡 - 高四位为 0：代表不保存到 SD 卡

【返回值】

无

【注意事项】

具体行为需 DSP 侧配合说明。

3.1.18 REFPARAM_GetDspCardLogFlag

【函数功能】

获取 DSP Log 是否保存到 SD 卡。

【函数原型】

```
PUBLIC uint8 REFPARAM_GetDspCardLogFlag(void) //Returns the dsp card log enable setting
//High 4-bit is DSP CARD LOG ENABLE: 0:Disable,
1:Enable
```

【参数说明】

无

【返回值】

DSP Log 是否保存到 SD 卡。

- 高四位为 1：代表保存到 SD 卡。
- 高四位为 0：代表不保存到 SD 卡

【注意事项】

具体行为需 DSP 侧配合说明。

3.1.19 SCI_SetAssertMode

【函数功能】

Assert 模式设置。

调用此 API 函数将设置 Assert 为 DEBUG 模式或 RELEASE 模式（即重启）。

【函数原型】

```
void SCI_SetAssertMode (  
    BOOLEAN    b_debug_mode  
);
```

【参数说明】

参数名称	含义
b_debug_mode	• SCI_TURE: 设置为 DEBUG 模式 • others: 设置为 RELEASE 模式

【返回值】

无

【注意事项】

无

3.1.20 SCI_SetArmLogFlag

【函数功能】

设置 Arm® Log 打开或关闭。

【函数原型】

```
PUBLIC void SCI_SetArmLogFlag (  
    BOOLEAN b_log  
);
```

【参数说明】

参数名称	含义
b_log	• SCI_TRUE: 打开 Log • SCI_FALSE: 关闭 Log

【返回值】

无

【注意事项】

无

3.1.21 SCI_GetArmLogFlag

【函数功能】

获取 Arm® Log 打开或关闭的信息。

【函数原型】

```
PUBLIC BOOLEAN SCI_GetArmLogFlag(void);
```

【参数说明】

无

【返回值】

- SCI_TRUE: Log 已打开。
- SCI_FALSE: Log 已关闭。

【注意事项】

无

3.1.22 SCI_ENGINEER_SetEnableArmSleepFlag

【函数功能】

打开或关闭 deepsleep 功能。

【函数原型】

```
PUBLIC void SCI_ENGINEER_SetEnableArmSleepFlag (uint32 mode);
```

【参数说明】

参数名称	含义
mode	• 1: 打开 deepsleep 功能 • 0: 关闭 deepsleep 功能

【返回值】

无

【注意事项】

无

3.1.23 SCI_ENGINEER_GetEnableArmSleepFlag

【函数功能】

获取 deepsleep 打开或关闭的信息。

【函数原型】

```
PUBLIC BOOLEAN SCI_ENGINEER_GetEnableArmSleepFlag (void);
```

【参数说明】

无

【返回值】

- SCI_TRUE: deepsleep 为打开。
- SCI_FALSE: deepsleep 为关闭。

【注意事项】

无

3.1.24 SIO_Close

【函数功能】

关闭指定的串口。

【函数原型】

```
void SIO_Close (  
uint32    port  
);
```

【参数说明】

参数名称	含义
port	• COM_DATA: 用户数据口 • COM_DEBUG: 调试口

【返回值】

无

【注意事项】

无

3.1.25 SIO_ChangeMode

【函数功能】

设置逻辑口 COM_DEBUG、COM_DATA 的模式。

- DEBUG PORT 一般用于 Log。
- USER PORT 可以用于 DATA_MODE、ATC_MODE 或者混合模式。对 USER PORT 模式的设置（在 UART 或者 VCOM）可以通过 API 函数 SIO_SetUserPortMode 实现。

【函数原型】

```
uint32 SIO_ChangeMode (
uint32 mode
);
```

【参数说明】

参数名称	含义
mode	设置模式

【返回值】

- SCI_SUCCESS：设置成功。
- SCI_ERROR：设置失败。

【注意事项】

无

3.1.26 SIO_SetUserPortMode

【函数功能】

设置用户口使用模式。

调用此 API 函数可将 UART（或者 VCOM）从 USER PORT 设置为 DATA_MODE、ATC_MODE 或者混合模式。

【函数原型】

```
uint32    SIO_SetUserPortMode (  
uint32    mode  
);
```

【参数说明】

参数名称	含义
mode	使用模式 - ATC_MODE: ATC 命令模式 - MIX_MODE: 混合模式, 即 AT 加上 GPRS 数据模式 - DATA_MODE: DATA 模式, 即 GPRS 数据模式

【返回值】

- SCI_SUCCESS: 设置成功。
- SCI_ERROR: 设置失败。

【注意事项】

无

3.1.27 SIO_DSPCardLogOpen

【函数功能】

开启 DSP Log 存放至 SD 卡功能。

【函数原型】

```
PUBLIC BOOLEAN SIO_DSPCardLogOpen(void)
```

【参数说明】

无

【返回值】

- TRUE: 设置成功。
- FALSE: 设置失败。

【注意事项】

此接口除了设置功能标志为 open 外, 还包含文件创建等动作。

3.1.28 SIO_DSPCardLogClose

【函数功能】

关闭 DSP Log 存放至 SD 卡功能。

【函数原型】

```
PUBLIC BOOLEAN SIO_DSPCardLogClose(void)
```

【参数说明】

无

【返回值】

- TRUE: 设置成功。
- FALSE: 设置失败。

【注意事项】

此接口除了设置功能标志为 close 外，还包含文件关闭等动作。

3.1.29 TM_GetSysDate

【函数功能】

获取系统日期。

【函数原型】

```
PUBLIC ERR_TM_E      TM_GetSysDate (  
SCI_DATE_T      *date_ptr  
);
```

【参数说明】

参数名称	含义
date_ptr	存放获取的系统日期，详细信息请参考 2.4.9 SCI_DATE_T 定义

【返回值】

获取系统日期的操作结果。

- ERR_TM_NONE: 获取成功。
- 其他: 获取失败。

详细信息请参考 [2.3.6 ERR_TM_E](#) 定义。

【注意事项】

无

3.1.30 TM_GetSysTime

【函数功能】

获取系统时间。

【函数原型】

```
PUBLIC ERR_TM_E      TM_GetSysTime (  
SCI_TIME_T          *time_ptr  
);
```

【参数说明】

参数名称	含义
time_ptr	存放获取的系统时间，详细信息请参考 2.4.10 SCI_TIME_T 定义

【返回值】

获取系统时间的操作结果。

- ERR_TM_NONE：获取成功。
- 其他：获取失败。

详细信息请参考 [2.3.6 ERR_TM_E](#) 定义。

【注意事项】

无

3.1.31 UBOT_SetDiskHidden

【函数功能】

设置设备隐藏状态。

【函数原型】

```
PUBLIC BOOLEAN UBOT_SetDiskHidden (  
uint8 *dev_name,  
BOOLEAN state
```

);

【参数说明】

参数名称	含义
dev_name	需要设置隐藏与否的设备
state	1: 隐藏 0: 可视

【返回值】

- TRUE: 设置成功。
- FALSE: 设置失败。

【注意事项】

无

3.1.32 UBOT_GetDiskHiddenState

【函数功能】

获取设备的隐藏状态。

【函数原型】

PUBLIC BOOLEAN UBOT_GetDiskHiddenState (uint8 *dev_name);

【参数说明】

参数名称	含义
dev_name	表示查询的设备

【返回值】

- TRUE: 隐藏。
- FALSE: 可视。

【注意事项】

无

3.1.33 VERSION_GetInfo

【函数功能】

获取软件版本信息。

【函数原型】

```
const char* VERSION_GetInfo(uint32 type);
```

【参数说明】

参数名称	含义
type	参考 2.3.10 VERSION_TYPE_E

【返回值】

指向软件版本信息字符串的指针。

【注意事项】

无

3.2 MMI 接口函数

3.2.1 MMIAPISET_SetTestModeEx

【函数功能】

设置网络类型。

【函数原型】

```
PUBLIC void MMIAPISET_SetTestModeEx(MN_DUAL_SYS_E dual_sys, uint8 *network_type);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2
network_type	需要设置的网络类型

【返回值】

无

【注意事项】

无

3.2.2 MMISRV_AUD_StartVibrate

【函数功能】

开始振动。

【函数原型】

```
PUBLIC MMISRV_HANDLE_T MMISRV_AUD_StartVibrate(uint16 pri, uint16 duation, uint16 interval, uint16 times, MMISRV_AUD_VIBRATE_CB callback);
```

【参数说明】

参数名称	含义
pri	音频播放的优先级
duation	一次振动的时长
interval	每次振动的间隔时长
times	振动次数
callback	振动结束后的回调

【返回值】

音频播放的唯一标识。

【注意事项】

无

3.2.3 MMISRV_AUD_StopVibrate

【函数功能】

停止振动。

【函数原型】

```
PUBLIC void MMISRV_AUD_StopVibrate(MMISRV_HANDLE_T handle);
```

【参数说明】

参数名称	含义
handle	音频播放的唯一标识

【返回值】

无

【注意事项】

无

3.2.4 MMI_GetAllNVSize

【函数功能】

获取所有模块 NV 总大小，包括 PS、REF、EFS 等。

【函数原型】

```
PUBLIC uint32 MMI_GetAllNVSize(void)
```

【参数说明】

无

【返回值】

所有模块 NV 总大小，单位 KB。

【注意事项】

无

3.2.5 MMI_GetAllModuleSumNvSize

【函数功能】

获取 MMI 所有模块的 NV 总大小。

【函数原型】

```
PUBLIC uint32 MMI_GetAllModuleSumNvSize(void);
```

【参数说明】

无

【返回值】

MMI 所有模块 NV 总大小，单位 KB。

【注意事项】

无

3.2.6 MMI_GetAllMMINVItem

【函数功能】

获取 MMI 所有模块 NV 项数目。

【函数原型】

```
PUBLIC uint32 MMI_GetAllMMINVItem(void)
```

【参数说明】

无

【返回值】

MMI 所有模块 NV 项数目。

【注意事项】

无

3.2.7 MMIENGARRAY_StrcatFromStrToUCS2

【函数功能】

设置有固定行列的函数的字符串显示。

【函数原型】

```
LOCAL uint16 MMIENGARRAY_StrcatFromStrToUCS2(netinfo_t netinfo, uint16 *uint16_str, uint16 *uint16_str_len)
```

【参数说明】

参数名称	含义
netinfo	指定当前含有行列内容的结构体
uint16_str	字符串
uint16_str_len	字符串的长度

【返回值】

字符串的长度。

【注意事项】

无

3.2.8 MMI_APIENG_HandlePlmnCnf

【函数功能】

读取 SIM 卡内置 UPLMN 列表并显示。

【函数原型】

```
PUBLIC MMI_RESULT_E MMI_APIENG_HandlePlmnCnf(DPARAM param)
```

【参数说明】

参数名称	含义
param	modem 侧上报的 UPLMN 列表信息，详细信息请参考《MN PHONE 模块接口使用指南》

【返回值】

- TRUE: 成功。
- FALSE: 失败。

【注意事项】

无

3.3 MN 接口函数

3.3.1 DMNV_SetUsimDrvLogFlag

【函数功能】

设置 SIM Trace 打开或关闭。

【函数原型】

```
BOOLEAN DMNV_SetUsimDrvLogFlag(BOOLEAN is_log_on);
```

【参数说明】

参数名称	含义
is_log_on	- SCI_TRUE: 打开 Log - SCI_FALSE: 关闭 Log

【返回值】

- TRUE: 成功。
- FALSE: 失败。

【注意事项】

无

3.3.2 mn_al_em_set_ims_parameter

【函数功能】

设置 IMS 的参数。

【函数原型】

```
BOOLEAN mn_al_em_set_ims_parameter(int32 dual_sys, uint16 param_id, char* value_ptr)
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2
param_id	设置 IMS 对应的参数 ID
value_ptr	设置 IMS 对应参数 ID 对应的值

【返回值】

- TRUE: 设置参数成功。
- FALSE: 设置参数失败。

【注意事项】

无

3.3.3 mn_al_em_get_ims_parameter

【函数功能】

获取 IMS 的参数。

【函数原型】

```
BOOLEAN mn_al_em_get_ims_parameter(int32 dual_sys,uint16 param_id, char* value_ptr)
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2
param_id	获取 IMS 对应的参数 ID
value_ptr	获取 IMS 对应参数 ID 对应的值

【返回值】

- TRUE: 获取参数成功。
- FALSE: 获取参数失败。

【注意事项】

无

3.3.4 MNPHONE_GetTestModeEx

【函数功能】

获取当前双卡的 TestMode。

【函数原型】

```
void MNPHONE_GetTestModeEx(uint8 *testmode)
```

【参数说明】

参数名称	含义
testmode	返回双卡的 TestMode

【返回值】

无

【注意事项】

无

3.3.5 LTE_Get_UE_Category_Info_Proc

【函数功能】

查询 LTE Category 能力。

【函数原型】

```
uint8 LTE_Get_UE_Category_Info_Proc(void);
```

【参数说明】

无

【返回值】

返回值跟 NV 配置有关，如果配置 Cat1，则返回 1，如果配置 Cat4，则返回 4。无效则返回 255。

【注意事项】

设备需要支持 LTE 模式，需要在协议栈激活后再查询，否则会返回无效值 255。例如：飞行模式下查询，则返回 255。

3.3.6 MNPHONE_GetCurrentLteRat

【函数功能】

返回当前的卡号对应的 RAT 是否支持 LTE。

【函数原型】

```
uint8 MNPHONE_GetCurrentLteRat(int32 dual_sys)
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2

【返回值】

- 非 0：当前卡号支持 LTE。
- 0：当前卡号不支持 LTE。

【注意事项】

无

3.3.7 MNPHONE_GetCurrentUmtsRat

【函数功能】

返回当前 RAT 是否支持 WCDMA。

【函数原型】

```
uint8 MNPHONE_GetCurrentUmtsRat(void)
```

【参数说明】

无

【返回值】

- 非 0：支持 WCDMA。
- 0：不支持 WCDMA。

【注意事项】

无

3.3.8 MNPHONE_Get_Fdd_Freqitem

【函数功能】

获取锁定 WCDMA 的频点和小区信息

【函数原型】

```
void MNPHONE_Get_Fdd_Freqitem(uint8* is_lock_freq, uint8* freqindex, MN_EM_LOCK_FREQ_CELL_T* lock_freq_info_ptr)
```

【参数说明】

参数名称	含义
is_lock_freq	返回当前是否进行锁定频点 • 1：是 • 0：否

参数名称	含义
freqindex	返回锁定的频点信息
lock_freq_info_ptr	返回锁定的频点和小区

【返回值】

无

【注意事项】

无

3.3.9 MNPHONE_SetLockedFreqAndCell

【函数功能】

设置锁定除 WCDMA 之外的频点和小区。

【函数原型】

```
void MNPHONE_SetLockedFreqAndCell(MN_DUAL_SYS_E dual_sys, MN_EM_LOCK_FREQ_CELL_T set_value);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2
set_value	锁定的频点和小区

【返回值】

无

【注意事项】

无

3.3.10 MNPHONE_GetLockedFreqAndCell

【函数功能】

获取除 WCDMA 之外锁定的频点和小区。

【函数原型】

```
MN_EM_LOCK_FREQ_CELL_T MNPHONE_GetLockedFreqAndCell( void )
```

【参数说明】

无

【返回值】

锁定的频点与小区信息。

【注意事项】

无

3.3.11 MNPHONE_UnlockedFreqAndCell

【函数功能】

解除锁定的频点和小区。

【函数原型】

```
void MNPHONE_UnlockedFreqAndCell( MN_DUAL_SYS_E dual_sys )
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2

【返回值】

无

【注意事项】

无

3.3.12 em_set_aoc_nv_param

【函数功能】

设置 AoC 计费功能的 NV 开关。

【函数原型】

```
void em_set_aoc_nv_param(uint8 aoc_enable);
```

【参数说明】

参数名称	含义
aoc_enable	0: 关闭 AoC 计费功能的 NV 开关 1: 打开 AoC 计费功能的 NV 开关

【返回值】

无

【注意事项】

无

3.3.13 em_get_aoc_nv_param

【函数功能】

获取 AoC 计费功能的开关状态。

【函数原型】

```
void em_get_aoc_nv_param(uint8 *aoc_enable);
```

【参数说明】

参数名称	含义
aoc_enable	返回 AoC 计费功能的开关状态。 0: AoC 计费功能的 NV 开关处于关闭状态 1: AoC 计费功能的 NV 开关处于打开状态

【返回值】

无

【注意事项】

无

3.3.14 MNSIM_GetAocEx

【函数功能】

获取当前 AoC 计费相关信息。

【函数原型】

```
MNSIM_AOC_T MNSIM_GetAocEx(MN_DUAL_SYS_E dual_sys);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2

【返回值】

当前 AoC 计费相关信息，具体参考 [2.4.6 MNSIM_AOC_T](#)。

【注意事项】

无

3.3.15 RRENG_Is_Delay_CSFB_1_Enable_upper

【函数功能】

获取 delay_type_1 的值。

【函数原型】

```
BOOLEAN RRENG_Is_Delay_CSFB_1_Enable_upper(void);
```

【参数说明】

无

【返回值】

- TRUE: delay_type_1=1。
- FALSE: delay_type_1=0。

【注意事项】

无

3.3.16 RRENG_Is_Delay_CSFB_2_Enable_upper

【函数功能】

获取 delay_type_2 的值。

【函数原型】

```
BOOLEAN RRENG_Is_Delay_CSFB_2_Enable_upper(void);
```

【参数说明】

无

【返回值】

- TRUE: delay_type_2=1。
- FALSE: delay_type_2=0。

【注意事项】

无

3.3.17 RRENG_Delay_CSFB_Enable

【函数功能】

激活 csfb_1_delay 或者 csfb_2_delay。

【函数原型】

```
void RRENG_Delay_CSFB_Enable(uint8 type);
```

【参数说明】

参数名称	含义
type	• 1: csfb_1_DelayEnable=1 • 不为 1: csfb_2_DelayEnable=1

【返回值】

无

【注意事项】

无

3.3.18 RRENG_Delay_CSFB_Disable

【函数功能】

去激活 csfb_1_delay 或者 csfb_2_delay。

【函数原型】

```
void RRENG_Delay_CSFB_Disable(uint8 type);
```

【参数说明】

参数名称	含义
type	1: csfb_1_DelayEnable=0 - 不为 1: csfb_2_DelayEnable=0

【返回值】

无

【注意事项】

无

3.3.19 em_get_fdd_support_band

【函数功能】

返回 WCDMA 支持的 BAND。

【函数原型】

```
void em_get_fdd_support_band (uint8 *band);
```

【参数说明】

参数名称	含义
band	返回 WCDMA 支持的 BAND

【返回值】

无

【注意事项】

无

3.3.20 em_al_get_locked_lte_multiband_for_scan

【函数功能】

获取锁定的 LTE BAND。

【函数原型】

```
BOOLEAN em_al_get_locked_lte_multiband_for_scan(  
    uint32 *locked_band_tdd_ptr,  
    uint32 *locked_band_fdd_ptr,  
    uint32 *locked_band_fdd_65_96_ptr  
)
```

【参数说明】

参数名称	含义
locked_band_tdd_ptr	返回锁定的 LTE TDD BAND
locked_band_fdd_ptr	返回锁定的 LTE FDD BAND
locked_band_fdd_65_96_ptr	返回锁定的 LTE FDD BAND 为 65 ~ 96

【返回值】

- TRUE：成功。
- FALSE：失败。

【注意事项】

无

3.3.21 em_al_get_locked_lte_multiband

【函数功能】

获取锁定的 LTE BAND。

【函数原型】

```
BOOLEAN em_al_get_locked_lte_multiband(
    uint32 *locked_band_tdd_33_64_ptr,
    uint32 *locked_band_fdd_1_32_ptr,
    uint32 *locked_band_fdd_65_80_ptr
)
```

【参数说明】

参数名称	含义
locked_band_tdd_33_64_ptr	返回锁定的 LTE TDD BAND 为 33 ~ 64
locked_band_fdd_1_32_ptr	返回锁定的 LTE FDD BAND 为 1 ~ 32
locked_band_fdd_65_80_ptr	返回锁定的 LTE FDD BAND 为 65 ~ 80

【返回值】

- TRUE: 成功。
- FALSE: 失败。

【注意事项】

无

3.3.22 MNPHONE_Set_LTE_Band

【函数功能】

设置锁定的 LTE BAND。

【函数原型】

```
BOOLEAN MNPHONE_Set_LTE_Band(uint32 *lte_band_ptr)
```

【参数说明】

参数名称	含义
lte_band_ptr [0]	设置锁定的 LTE FDD BAND 在 1 ~ 32
lte_band_ptr [1]	设置锁定的 LTE TDD BAND 在 33 ~ 64
lte_band_ptr [2]	设置锁定的 LTE FDD BAND 在 65 ~ 80

【返回值】

- TRUE: 设置锁定的 LTE BAND 成功。
- FALSE: 设置锁定的 LTE BAND 失败。

【注意事项】

无

3.3.23 MNPHONE_Get_LTE_Band

【函数功能】

获取锁定的 LTE BAND。

【函数原型】

```
BOOLEAN MNPHONE_Get_LTE_Band(uint32 lte_band[3])
```

【参数说明】

参数名称	含义
lte_band[0]	返回锁定的 LTE FDD BAND 为 1 ~ 32
lte_band[1]	返回锁定的 LTE TDD BAND 为 33 ~ 64
lte_band[2]	返回锁定的 LTE FDD BAND 为 65 ~ 80

【返回值】

- TRUE: 获取锁定的 LTE BAND 成功。
- FALSE: 获取锁定的 LTE BAND 失败。

【注意事项】

无

3.3.24 MNPHONE_SetRAT_For_Lock

【函数功能】

设置指定 SIM 卡上的 TestMode 是否进行锁定。

【函数原型】

```
BOOLEAN MNPHONE_SetRAT_For_Lock(MN_DUAL_SYS_E dual_sys, uint8 testmode_value, BOOLEAN is_lock);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2
testmode_value	当前卡号对应的 TestMode
is_lock	设置是否进行锁定 1: 锁定 0: 不锁定

【返回值】

- TRUE: 设置成功。
- FALSE: 设置失败。

【注意事项】

无

3.3.25 em_set_fdd_rf_band

【函数功能】

根据 STATUS，设置 WCDMA 的 BAND。

【函数原型】

```
BOOLEAN em_set_fdd_rf_band(uint8 band, uint8 status)
```

【参数说明】

参数名称	含义
band	频段
status	1: 如果之前已经设置过该 BAND，不做任何处理，否则对该 BAND 进行设置。 0: 如果之前已经设置过该 BAND，删除该 BAND，否则不做任何处理。

【返回值】

- TRUE: 设置该 BAND 成功。
- FALSE: 设置该 BAND 失败。

【注意事项】

无

3.3.26 em_get_fdd_rf_band

【函数功能】

获取 WCDMA 的 RF BAND。

【函数原型】

```
BOOLEAN em_get_fdd_rf_band(uint band, uint8 *status);
```

【参数说明】

参数名称	含义
band	频段
status	返回设置 BAND 的状态。 1: 已经设置过该 BAND 0: 没有设置过该 BAND

【返回值】

- TRUE: 获取该 BAND 成功。
- FALSE: 获取该 BAND 失败。

【注意事项】

无

3.3.27 MNPHONE_SelectBandEx

【函数功能】

用来设置工作频段，可选择单频或双频。是一个异步调用的 API。

【函数原型】

```
ERR_MNPHONE_CODE_E MNPHONE_SelectBandEx (
    MN_DUAL_SYS_E      dual_sys,
    MN_PHONE_BAND_TYPE_E user_band
);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2
user_band	选择的频段必须是硬件支持的频段，支持情况可以通过 CLASSMARK_GetPGsmSupport 等函数查询

【返回值】

- ERR_MNPHONE_NO_ERR: 调用 API 成功。
- 其他: 调用失败。详细信息请参考 [2.3.3 ERR_MNPHONE_CODE_E](#) 定义。

因为是异步调用，所以该操作执行完后会产生事件 EV_MN_APP_SET_BAND_CNF_F。

如果使用的是缺省的回调函数，会收到消息 APP_MN_SET_BAND_CNF，消息会带有 PIN 操作结果和 PIN 当前状态的参数，可以参考结构 APP_MN_SET_BAND_CNF_T。

【注意事项】

无

3.3.28 MNPHONE_GetBandEx

【函数功能】

用来获取工作频段，单频或双频。是一个同步调用的 API。

【函数原型】

```
MN_PHONE_BAND_TYPE_E    MNPHONE_GetBandEx (
MN_DUAL_SYS_E    dual_sys
);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2

【返回值】

返回工作频段类型，单频或双频。详细信息请参考 [2.3.8 MN_PHONE_BAND_TYPE_E](#) 定义。

【注意事项】

无

3.3.29 MNPHONE_GprsAttachEx

【函数功能】

发起 GPRS 的 Attach 请求消息。

【函数原型】

```
ERR_MNPHONE_CODE_E MNPHONE_GprsAttachEx(
    MN_DUAL_SYS_E dual_sys,
    MN_PHONE_GPRS_ATTACH_TYPE_E mn_attach_type
);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2
mn_attach_type	Attach GPRS 的方式，联合或 GPRS only

【返回值】

- ERR_MNPHONE_NO_ERR: 调用 API 成功。
- 其他: 调用失败。详细信息请参考 [2.3.3 ERR_MNPHONE_CODE_E](#) 定义。

【注意事项】

无

3.3.30 MNPHONE_GprsDetachEx

【函数功能】

发起 GPRS 的 Detach 请求消息。

【函数原型】

```
ERR_MNPHONE_CODE_E MNPHONE_GprsDetachEx(
    MN_DUAL_SYS_E dual_sys,
    MN_PHONE_GPRS_DETACH_TYPE_E mn_detach_type
);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2
mn_detach_type	Detach GPRS 的方式，联合或 GPRS only

【返回值】

- ERR_MNPHONE_NO_ERR: 调用 API 成功。
- 其他：调用失败。详细信息请参考 [2.3.3 ERR_MNPHONE_CODE_E](#) 定义。

【注意事项】

无

3.3.31 Lte_SetAntennaSingleMode

【函数功能】

设置 LTE 天线模式，通过 NV 项 pLTEDownload_Dspcompensation_CC0_optimize->LTE_Flag_config[59]的值实现。

【函数原型】

```
void Lte_SetAntennaSingleMode(uint32 ulMode);
```

【参数说明】

参数名称	含义
ulMode	0: LTE 设置为双天线 1: LTE 设置为单天线

【返回值】

无

【注意事项】

无。

3.3.32 Lte_GetAntennaSingleMode

【函数功能】

获取 LTE 天线模式，通过 NV 项 pLTEDownload_Dspcompensation_CC0_optimize->LTE_Flag_config[59]的值实现。

【函数原型】

```
uint32 Lte_GetAntennaSingleMode(void);
```

【参数说明】

无

【返回值】

- 该 NV 项为 0 表示 LTE 设置为双天线。
- 该 NV 项为 1 表示 LTE 设置为单天线。

【注意事项】

无。

3.3.33 Lte_SetAntennaMode

【函数功能】

设置 LTE 主辅天线，通过 NV 项 pLTEDownload_Dspcompensation_CC0_toolcomp->LTE_reserved[1][0]的值实现。

【函数原型】

```
void Lte_SetAntennaMode(uint32 ulMode);
```

【参数说明】

参数名称	含义
ulMode	• 1 或者 0x40: 表示 LTE 主天线 only • 2 或者 0x10: 表示 LTE 辅天线 only

【返回值】

无

【注意事项】

无。

3.3.34 Lte_GetAntennaMode

【函数功能】

获取 LTE 主辅天线，通过 NV 项 pLTEDownload_Dspcompensation_CC0_toolcomp->LTE_reserved[1][0]的值实现。

【函数原型】

```
uint32 Lte_GetAntennaMode(void);
```

【参数说明】

无

【返回值】

- 为 1 或者 0x40：表示 LTE 主天线 only。
- 为 2 或者 0x10：表示 LTE 辅天线 only。

【注意事项】

无。

3.3.35 MNPHONE_SetServiceTypeEx

【函数功能】

用来设置用户要求的服务类型，可以仅支持 GSM，也可以同时支持 GSM 和 GPRS。是一个异步调用的 API。

【函数原型】

```
ERR_MNPHONE_CODE_E MNPHONE_SetServiceTypeEx (  
MN_DUAL_SYS_E          dual_sys,  
MN_PHONE_USER_SERVICE_SELECT_E service_type  
);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2
service_type	用户要求的服务类型，可以仅支持 GSM，也可以同时支持 GSM 和 GPRS

【返回值】

- ERR_MNPHONE_NO_ERR: 调用 API 成功。
- 其他: 调用失败。详细信息请参考 [2.3.3 ERR_MNPHONE_CODE_E](#) 定义。

【注意事项】

无

3.3.36 MNPHONE_GetServiceTypeEx

【函数功能】

获取用户要求的服务类型，可以仅支持 GSM，也可以同时支持 GSM 和 GPRS。是一个同步调用的 API。

【函数原型】

```
MN_PHONE_USER_SERVICE_SELECT_E MNPHONE_GetServiceTypeEx (
MN_DUAL_SYS_E    dual_sys
);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2

【返回值】

函数将立即返回用户要求的服务类型，是仅支持 GSM，还是同时支持 GSM 和 GPRS。

【注意事项】

无

3.3.37 MNPHONE_GetBand

【函数功能】

用来获取工作频段：单频或双频。是一个同步调用的 API。

【函数原型】

```
MN_PHONE_BAND_TYPE_E MNPHONE_GetBand (void);
```

【参数说明】

无

【返回值】

工作频段，单频或双频。详细信息请参考 [2.3.8 MN_PHONE_BAND_TYPE_E](#) 枚举定义。

【注意事项】

无

3.3.38 MNPHONE_SetServiceType

【函数功能】

用来设置用户要求的服务类型，可以仅支持 GSM，也可以同时支持 GSM 和 GPRS。是一个异步调用的 API。

【函数原型】

```
ERR_MNPHONE_CODE_E MNPHONE_SetServiceType (
    MN_PHONE_USER_SERVICE_SELECT_E service_type
);
```

【参数说明】

参数名称	含义
service_type	用户要求的服务类型。可以仅支持 GSM，也可以同时支持 GSM 和 GPRS。

【返回值】

- ERR_MNPHONE_NO_ERR: 调用 API 成功。

- 其他：调用失败。详细信息请参考 [2.3.3 ERR_MNPHONE_CODE_E](#) 定义。

【注意事项】

无

3.3.39 MNPHONE_GetServiceType

【函数功能】

获取用户要求的服务类型，可以仅支持 GSM，也可以同时支持 GSM 和 GPRS。是一个同步调用的 API。

【函数原型】

```
MN_PHONE_USER_SERVICE_SELECT_E MNPHONE_GetServiceType (void);
```

【参数说明】

无

【返回值】

API 将立即返回用户要求的服务类型，是仅支持 GSM，还是同时支持 GSM 和 GPRS。

【注意事项】

无

3.3.40 MNNV_GetIMEI

【函数功能】

获取 IMEI 的值。

【函数原型】

```
BOOLEAN MNNV_GetIMEI (
    MN_IMEI_T imei
);
```

【参数说明】

参数名称	含义
imei	存放 IMEI 值的结果

【返回值】

- TRUE: 读取 IMEI 值成功。
- FALSE: 读取 IMEI 值失败。

【注意事项】

无

3.3.41 MNNV_GetIMEIEx

【函数功能】

获取 IMEI。是一个同步调用的 API

【函数原型】

```
BOOLEAN      MNNV_GetIMEIEx (
MN_DUAL_SYS_E    dual_sys,
MN_IMEI_T        imei
);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 • 0: 卡 1 • 1: 卡 2
imei	设备的 IMEI 在该参数中返回

【返回值】

- TRUE: 获取 IMEI 成功。
- FALSE: 获取 IMEI 失败。

【注意事项】

无

3.3.42 MNPHONE_GetMeasReportEx

【函数功能】

获取临近小区信息。是一个同步调用的 API。

【函数原型】

```
ERR_MNPHONE_CODE_E MNPHONE_GetMeasReportEx (
    MN_DUAL_SYS_E      dual_sys,
    APP_MN_NCELL_RSSI_CNF_T *ncell_meas_ptr
);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡。 0: 卡 1。 1: 卡 2。
ncell_meas_ptr	保存获取到的临近小区信号信息指针，包括信号强度、小区 ID 等。详细信息请参考 2.4.2 APP_MN_NCELL_RSSI_CNF_T 定义。

【返回值】

- ERR_MNPHONE_NO_ERR: 调用 API 成功。
- 其他: 调用失败。详细信息请参考 [2.3.3 ERR_MNPHONE_CODE_E](#) 定义。

【注意事项】

无

3.3.43 MNPHONE_GetRSSIEx

【函数功能】

获取服务小区信息。是一个同步调用的 API。

【函数原型】

```
MN_PHONE_SCELL_MEAS_T MNPHONE_GetRSSIEx (
    MN_DUAL_SYS_E      dual_sys
);
```

【参数说明】

参数名称	含义
dual_sys	指定当前使用的 SIM 卡 0: 卡 1 1: 卡 2

【返回值】

直接返回服务小区的基本信息，包括信号强度和频点等信息。详细信息请参考 [2.4.5 MN_PHONE_SCELL_MEAS_T](#) 定义。

【注意事项】

无

3.4 L1 接口函数

3.4.1 L1API_GetAFC

【函数功能】

获取当前的 AFC 值。

【函数原型】

```
BOOLEAN L1API_GetAFC (
uint32 task_id
);
```

【参数说明】

参数名称	含义
task_id	调用任务的 ID

【返回值】

- SCI_TRUE: 操作成功。
- SCI_FALSE: 操作失败，上次任务未完成。

Layer1 回复消息 APP_L1_GET_AFC_CNF。

【注意事项】

无

3.4.2 L1API_GetPA

【函数功能】

获取当前的 PA 值。

【函数原型】

```
BOOLEAN    L1API_GetPA (  
uint32     task_id  
);
```

【参数说明】

参数名称	含义
task_id	调用任务的 ID

【返回值】

- SCI_TRUE: 操作成功。
- SCI_FALSE: 操作失败, 上次任务未完成。

Layer1 回复消息 APP_L1_GET_PA_CNF。

【注意事项】

无

3.4.3 L1API_GetAGC

【函数功能】

获取当前的 AGC 值。

【函数原型】

```
BOOLEAN    L1API_GetAGC (  
uint32     task_id  
);
```

【参数说明】

参数名称	含义
task_id	调用任务的 ID

【返回值】

- SCI_TRUE: 操作成功。
- SCI_FALSE: 操作失败, 上次任务未完成。

Layer1 回复消息 APP_L1_GET_AGC_CNF。

【注意事项】

无

3.4.4 L1API_ApplyMaxPower

【函数功能】

使用最大功率发送。

上层调用该函数请求使用最大功率发送，调用该函数后需要重新启动才能恢复 PA 到正常。

【函数原型】

```
BOOLEAN L1API_ApplyMaxPower(void);
```

【参数说明】

无。

【返回值】

SCI_TRUE：操作成功。（该接口返回值固定）

【注意事项】

无

3.4.5 L1API_WriteGsmDivState

【函数功能】

AT 命令控制辅路天线状态。

【函数原型】

```
BOOLEAN L1API_WriteGsmDivState(uint16 div_state);
```

【参数说明】

参数名称	含义
div_state	辅路状态 0：单主路，辅路关闭 1：动态开辅路 2：强制合并辅路 3：单辅路，主路关闭

【返回值】

- TRUE: 操作成功。
- FALSE: 操作失败。

【注意事项】

无

3.4.6 L1API_ReadGsmDivState

【函数功能】

AT 命令读取辅路天线状态。

【函数原型】

```
BOOLEAN L1API_ReadGsmDivState(uint16* div_state);
```

【参数说明】

参数名称	含义
div_state	辅路状态 • 0: 单主路，辅路关闭 • 1: 动态开辅路 • 2: 强制合并辅路 • 3: 单辅路，主路关闭

【返回值】

- TRUE: 操作成功。
- FALSE: 操作失败。

【注意事项】

首次调用会返回 NV 中实际获取的值，之后每次调用则会返回上次设置的该参数值。

3.4.7 WL1C_RF_Set_Path_For_ATNA_Test

【函数功能】

支持双天线的设备在 WCDMA 模式下设置天线工作在单独主路、单独辅路或主辅都开。

【函数原型】

```
BOOLEAN WL1C_AT_Set_Path_For_ATNA_Test(_u32 cmd)
```

【参数说明】

参数名称	含义
cmd	- cmd=0: 单独主路 - cmd=17: 单独辅路 - cmd=1: 主辅都开

【返回值】

- TRUE: 操作成功。
- FALSE: 操作失败。

【注意事项】

无

3.4.8 WL1C_RF_Get_Path_For_ATNA_Test

【函数功能】

查询 WCDMA 模式下天线工作模式。

【函数原型】

```
_u16 WL1C_AT_Get_Path_For_ATNA_Test(void)
```

【参数说明】

无

【返回值】

- cmd=0: 单独主路
- cmd=17: 单独辅路
- cmd=1: 主辅都开

【注意事项】

无

3.4.9 LAYER1_GetLayer1MonItem

【函数功能】

AT 命令查看 Search 和 SCH 成功的次数。

【函数原型】

```
void LAYER1_GetLayer1MonItem(  
    LAYER1_MONITOR_ITEM_T *item_ptr  
);
```

【参数说明】

参数名称	含义
item_ptr	Search 和 SCH 成功的次数

【返回值】

无

【注意事项】

无

3.4.10 DRV_RF_NV_Support_Diversity_Path

【函数功能】

查询 WCDMA 模式下是否支持双天线

【函数原型】

```
_u32 DRV_RF_NV_Support_Diversity_Path(void)
```

【参数说明】

无

【返回值】

- TRUE: UE 支持双天线
- FALSE: UE 不支持双天线

【注意事项】

无

3.5 TCPIP 接口函数

3.5.1 TCPIPDEBUG_EnablePktOutput

【函数功能】

控制 cap output 开关。

【函数原型】

```
PUBLIC void TCPIPDEBUG_EnablePktOutput( BOOLEAN is_enable );
```

【参数说明】

参数名称	含义
is_enable	- TRUE: 打开 - FALSE: 关闭

【返回值】

无

【注意事项】

无

3.5.2 TCPIPDEBUG_IsPktOutputEnable

【函数功能】

获取 cap output 开关。

【函数原型】

```
PUBLIC BOOLEAN TCPIPDEBUG_IsPktOutputEnable( void );
```

【参数说明】

无

【返回值】

- TRUE: 打开。
- FALSE: 关闭。

【注意事项】

无

4 参考文档

《MN PHONE 模块接口使用指南》